PROGRESS SUMMARY

Automated Floor-Plan Generation Pipeline

From an Excel program table to a resolved, multi-floor plan with rooms, corridors, and doors

Rhino 8 · Grasshopper · CPython | Pilot / R&D | Prepared for design-team review

Current status: Parts 0–3 complete — reader, bubble solver, tiler, corridor organizer, corridor-width, and door passes all working on the validated dataset. Part 4 (vertical cores) is the next build.

1. What the tool does

The pipeline turns an architectural program — the spreadsheet of required rooms a team already produces — into a laid-out floor plan. It reads the program, arranges rooms by their required adjacencies, resolves them into real rectangles grouped by department, then threads a corridor network and places a door for every room. The intent is speed and breadth at the early planning stage: generate several complete, comparable plan options in the time it currently takes to block out one by hand, then hand the chosen option to an architect to refine.

It runs as six self-contained components in sequence. A designer owns the two ends — the program going in, and the judgment applied to the options coming out. Everything in between runs automatically.

The pipeline at a glance

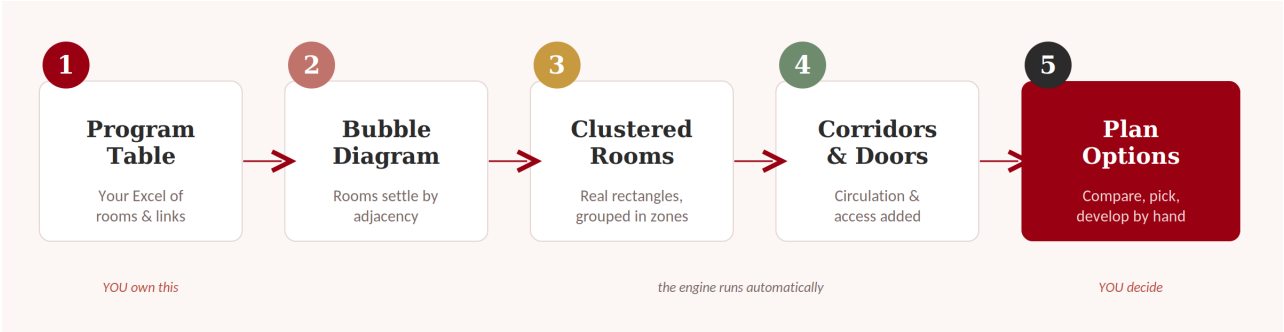


Figure 1 — The five-stage flow. The design team owns the program going in and the judgment applied to the options coming out.

#	Stage	What it produces
01	Program Reader	Reads the Excel sheet; expands quantities into individual room instances with their adjacencies
02	Bubble Solver	Arranges rooms as area-sized bubbles that pull together by adjacency — a generated adjacency diagram
03	Cluster Tiler	Hardens bubbles into exact-area rectangles, grouped into department clusters
04	Corridor Organizer	Re-lays rooms into compact double-loaded rows and lays out the corridor skeleton
05	Corridor Width Pass	Gives corridors real widths — main spines widen with the traffic they carry
06	Door Pass	Places a door for every room, including direct doors between rooms that share access

2. Current state

All six components are built and validated end-to-end against a real test program. The system reliably produces a complete plan — rooms, corridors, and doors across multiple floors — from a single spreadsheet, and it does so repeatably enough to generate variations for comparison.

Validated on a real dataset

The pipeline was exercised on a school program (Program_ExcelToGrasshopper_Test.xlsm). The numbers below are what the tool actually produced, not targets:

Measure	Result
Program rows read	68 rows on Sheet2
Rooms placed	151 individual room instances after quantity expansion
Adjacencies wired	145 directed room-to-room connections
Departments / floors	Departments 1–9, across floors 1–2
Bubble overlap after solve	Effectively zero (0 overlapping pairs after the de-overlap pass)
Room area accuracy	Rooms hit their exact programmed area at shaping (0% deviation)
Cluster balance	Rooms distributed across clusters (e.g. 36 / 23 / 13 / 12 / 10 / 7 on floor 1)
Corridor widening	Main corridors scale with load — from 3 ft up to an 8 ft cap by rooms served

What works well

- The input is genuinely a plain spreadsheet. Four columns carry the load — what the room is, how big, how many, and what it connects to — so the design team can drive the tool without touching code.
- Rooms come out at their exact programmed areas, grouped into sensible department clusters, with the adjacency logic clearly driving who ends up next to whom.
- The corridor network is connected by construction: corridors are routed through channels reserved during layout, rather than computed and stitched together afterward. This removed the fragmentation that broke an earlier version of the pipeline.
- Corridors are hierarchical and traffic-aware — a main spine that serves more rooms is automatically drawn wider than a minor branch.
- The tool is fast enough to run repeatedly, which is the whole point: it produces options to compare rather than a single fixed answer.

Known limitations (current, honest state)

These are documented deliberately — the working principle on this project is to ship working features and record their limits rather than iterate indefinitely on fragile pieces.

Limitation	What it means	Why it happens / lever
Floor-space efficiency ~31–39%	A meaningful share of each floor reads as whitespace inside clusters.	The structural cost of insisting on exact areas, perfectly flat corridor-facing walls, and reserved corridor channels. Density levers exist (shallower rooms, more elongation, or relaxing exact area).
Vertical circulation not yet built	Stairs and elevators between floors aren't placed automatically yet.	This is Part 4, the next scheduled build. The solver already aligns cross-floor rooms 'as if stacked' to support it.
Shared-Access column must be typed text	Rooms that should connect directly to a neighbor (e.g. a restroom to a classroom) need an 'X' or 'TRUE' typed in the sheet.	Excel's clickable checkboxes aren't readable by the reader library; a typed value is required, or every room simply gets a corridor door.
A few program edges won't resolve	Adjacencies that point at a zero-quantity room are skipped.	Expected behavior — reported in diagnostics, not an error. Comes from the program data, not the tool.

3. Component walkthrough

Each stage produces a drawing the design team can review directly. Read them in sequence — the same program carried all the way through.

The underlying concept

The whole pipeline rests on one idea: a bubble diagram with adjacency links translates directly into rectangular rooms, doors, and a corridor hierarchy. Rooms cluster, clusters face a shared corridor, and corridors organize into branches feeding a trunk.

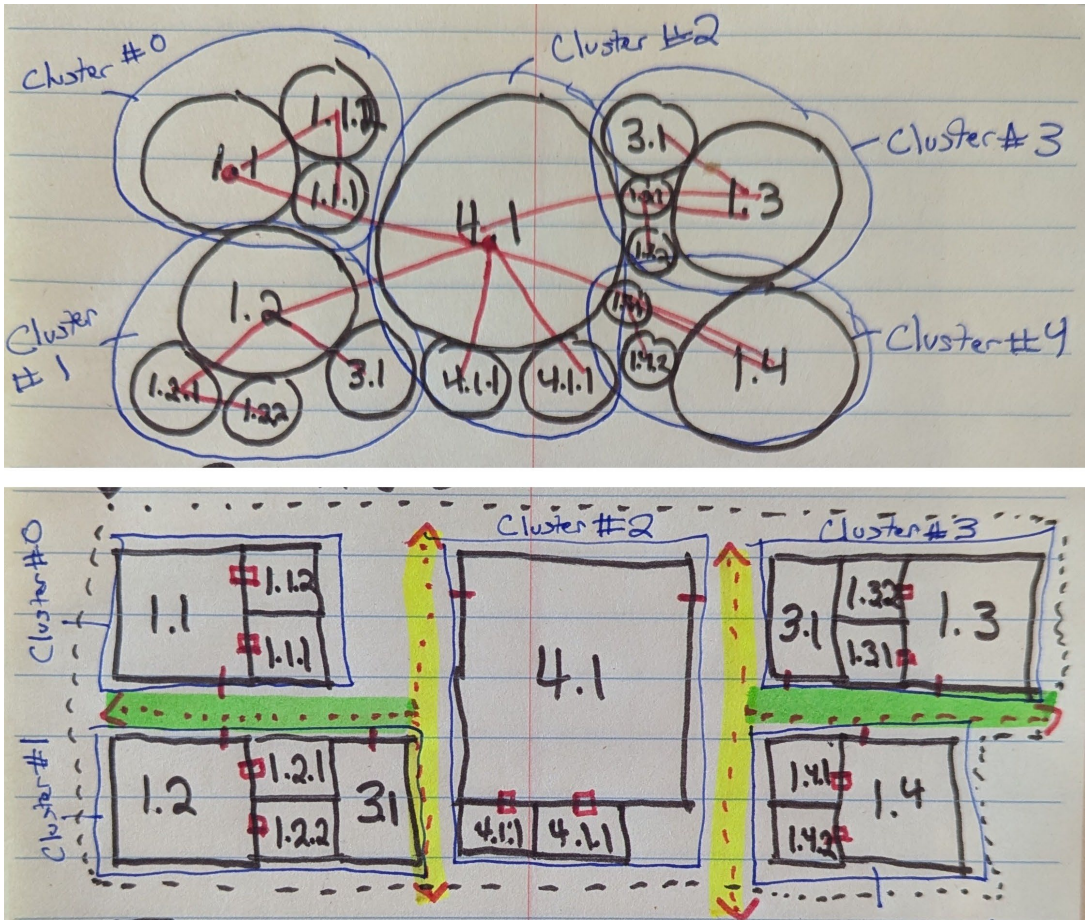


Figure 2 — The governing concept: bubbles with adjacency connections (above) resolve into rectangular rooms with direct adjacencies, branch corridors (green), and trunk corridors (yellow) below.

Component 01 — Program Reader (the input)

The only thing the design team supplies is the program spreadsheet. Four columns do the work: what the room is, how big it is, how many are needed, and what it connects to. The reader cleans the sheet, skips spacer and zero-quantity rows, and expands quantities into individual room instances — so a row for four classrooms becomes four separately placed rooms.

Program_ExcelToGrasshopper.xlsm — Sheet2

Dept No	Program Element	Room Qty	Net SF	Conn. Primary	Floor / Color
1.1	Classroom - K	4	900	4.1	1
1.2	Classroom - 1	4	900	4.1	1
4.1	Common Area	5	1200	—	1
6.1	Mini-Gym	1	4200	4.1	1
7.9	Clinic	1	650	4.1	1
2.1	Art	1	1100	4.1	2
	... 68 rows total				

Four columns do the work:

WHAT the room is · HOW BIG · HOW MANY · WHAT it connects to

No coding. If you can fill a spreadsheet, you can drive the tool.

Figure 3 — The program table as the tool reads it. No scripting is required on the design side.

Component 02 — Bubble Solver

Every room becomes a circle sized by its area. Required adjacencies act as springs pulling rooms together while repulsion keeps them from overlapping, and each floor is held together as a coherent group. The result reads like a hand-drawn adjacency diagram — but it is generated from the table, and it settles with effectively zero overlap.

- Bubble size = room area · lines = required adjacencies · color = department.

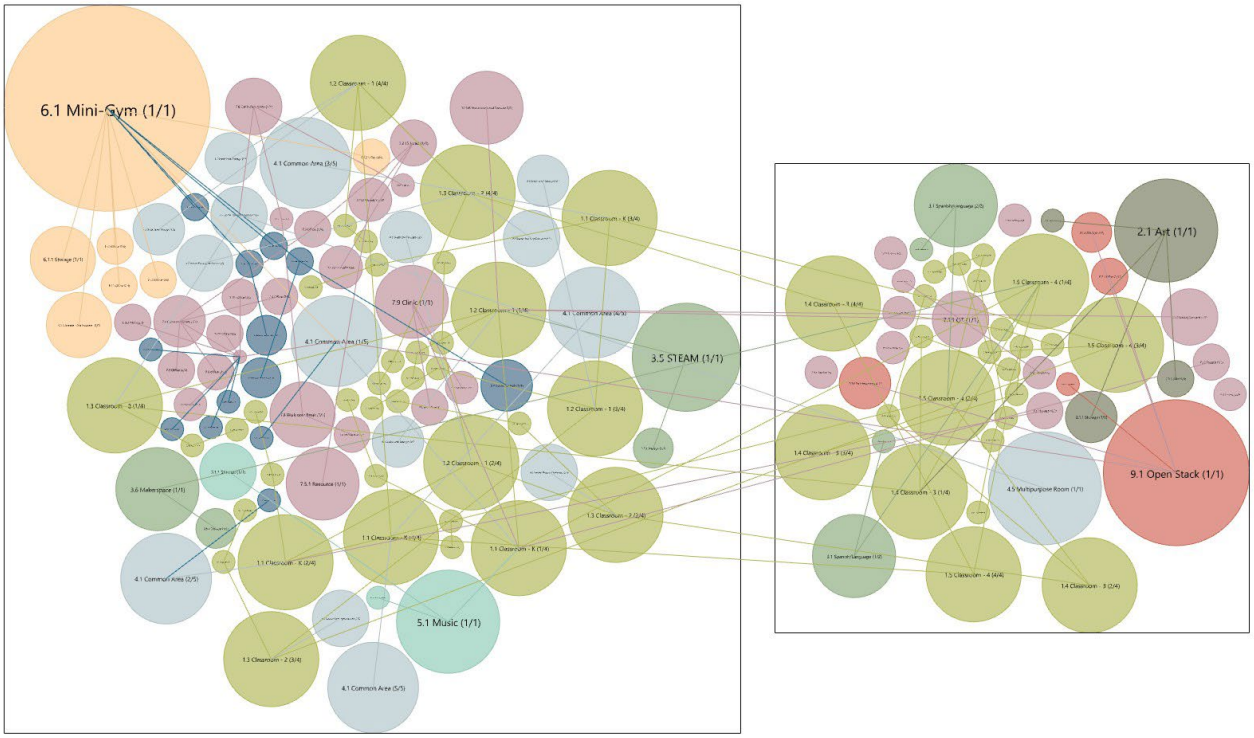


Figure 4 — Bubble diagram output. Floor 1 (left) and floor 2 (right); large program elements such as the Mini-Gym, Art, and Open Stack read immediately.

Component 03 — Cluster Tiler

The soft bubbles harden into rectangles at their exact programmed areas, grouped into department clusters and aligned into bands. Rooms are tagged with the wall that will face circulation, and channels are reserved between clusters so corridors have somewhere to run. It now reads as a plan, though no corridor geometry exists yet.

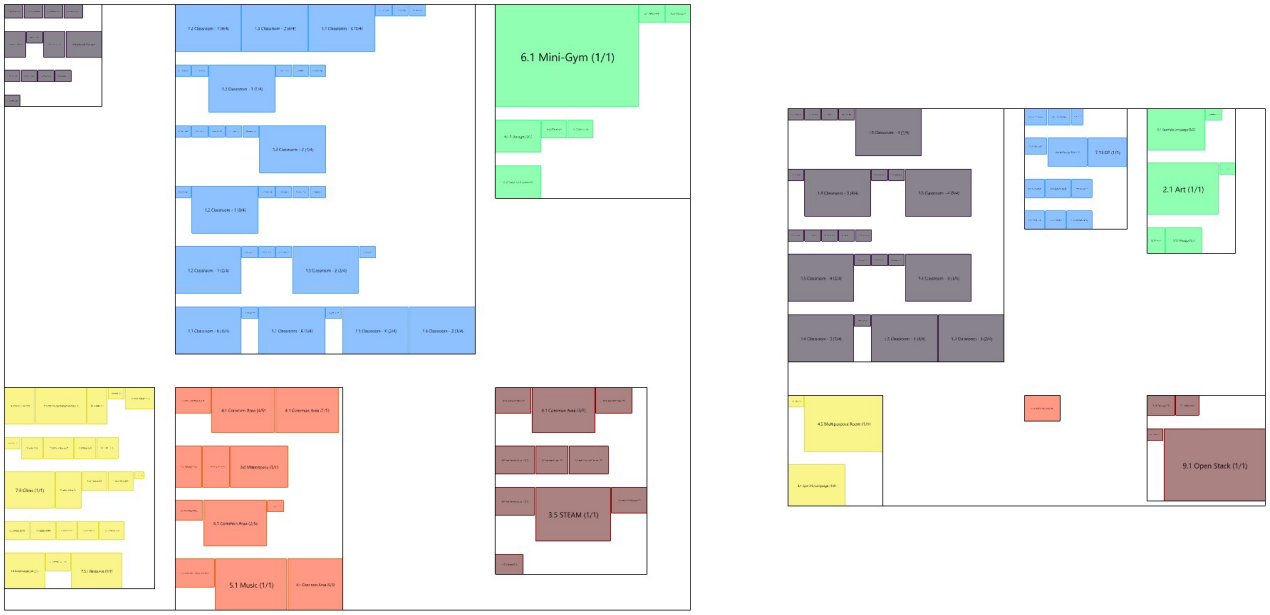


Figure 5 — Clustered rooms at exact area, grouped by department. Reserved gaps between clusters become the corridor channels.

Components 04–06 — Corridors, Widths, and Doors

The organizer re-lays the rooms into compact double-loaded rows — two rows of rooms facing a shared corridor — and emits the corridor skeleton. Because corridors are routed through channels reserved during layout rather than computed afterward, the network is connected by construction. The width pass then gives each corridor a real width, widening the main spines according to how many rooms they serve, and the door pass places a door on every room.

- Branch corridors run between the two room rows of a branch.
- Trunk corridors form the main spine, widening with traffic — from 3 ft up to an 8 ft cap.
- Doors sit at the wall midpoint facing the nearest corridor; rooms flagged for shared access instead get a direct door into their neighbor.



Figure 6 — The resolved plan: rooms in double-loaded rows with the branch and trunk corridor network threaded through both floors.

The result — options, not a single answer

Because the whole thing runs from one spreadsheet, changing a few settings and re-running yields a fresh, complete option. This is how the tool is meant to be used: generate several, compare them side by side, then pick one to develop by hand.

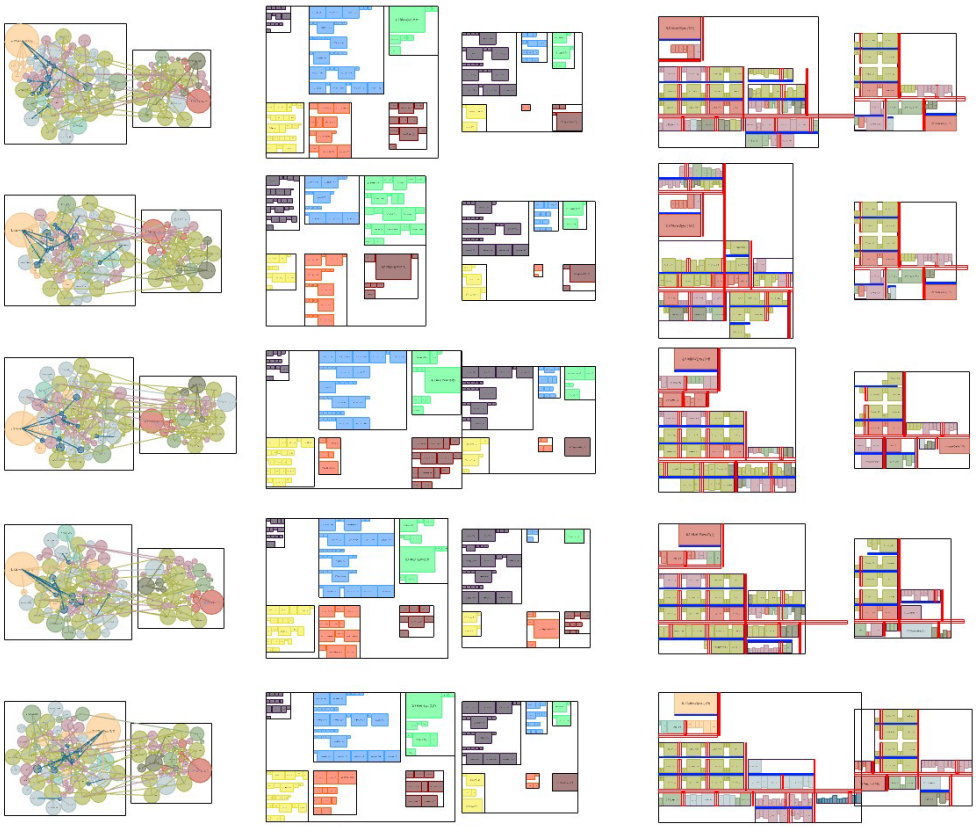


Figure 7 — Five generated variations of the same program, shown as bubbles, clustered rooms, and resolved corridor plans.

4. Future refinement and changes

Next build — Part 4: vertical cores

The immediate next step is automatic placement of stairs and elevators between floors, driven by cross-floor adjacency and department, each with reserved space. The groundwork is already in place: the bubble solver aligns cross-floor adjacencies as if the floors were stacked, so vertical cores can line up cleanly.

Density and efficiency

The most visible refinement target is floor-space efficiency. The tool already exposes the levers — reducing target room depth, allowing rooms to elongate more, or relaxing the exact-area constraint — and a focused pass on tuning these against real projects would tighten the plans. This is a trade-off study to run with the design team, since each lever changes the character of the result.

Program input experience

- Smooth over the Shared-Access typed-text requirement so it's less of a gotcha for people building the spreadsheet.
- Add light validation and clearer diagnostics so program issues (unresolved adjacencies, zero-quantity targets) surface in plain language as the sheet is built.

Design-review feedback to fold in

This pilot is being shown to the design team specifically to gather direction. The open questions that will shape the next round of changes:

- Does the clustering logic match how architects actually group space?
- Are corridors landing where a designer would expect them?
- What program inputs is the tool missing to be useful on a live project?
- Where would this genuinely save time — and where does hand-work still win?



Figure 8 — Review markup on a generated plan. Redundant and mis-placed corridor runs are flagged directly on the output; this is the feedback loop driving refinement.

Working principle going forward

> **Guiding rule: Ship** working features and document their limitations, rather than iterating indefinitely on fragile subsystems. The tool is a fast, credible starting point — designer judgment finishes the plan.